

คู่มือการพัฒนาโปรแกรม Gloop RVM

วิธีการติดตั้ง ตั้งค่า และทดสอบระบบ Smart Reverse Vending Machine

★ ภาพรวมของระบบและส่วนประกอบ

Web Application

Next.js 15

TypeScript

Firebase

โฟลเดอร์: `web/`

UI สำหรับผู้ใช้ — Login, Dashboard แสดงคะแนน real-time, Rewards, Profile
ทดสอบได้ทันทีโดยไม่ต้องมี Hardware

Python AI Service

Python 3.10+

YOLOv8

FastAPI

โฟลเดอร์: `ai_server/`

ดึง Firestore → วิเคราะห์ภาพขวดด้วย YOLO → เขียนผลกลับ
ทดสอบได้โดยอัปโหลดภาพผ่าน REST API

⚡ Firebase Cloud Functions

Node.js 22

Firebase

โฟลเดอร์: `functions/`

`uploadBottleImage` — รับภาพจาก ESP32, บันทึกใน Storage, ตั้ง status
Deploy บน Firebase (ไม่ต้อง run local)

🔧 ESP32 Firmware (Hardware)

Arduino C++

ESP32-S3

โฟลเดอร์: `esp32/`

Master: กล้อง label + solenoid | Slave: กล้อง cap
ต้องมีบอร์ดจริง — ดู Section ทดสอบโดยไม่มี Hardware

0 Software ที่ต้องติดตั้งก่อน (Prerequisites)

SOFTWARE	VERSION	วัตถุประสงค์	DOWNLOAD
Node.js	18 LTS หรือใหม่กว่า	Web App + Firebase CLI	nodejs.org
Python	3.10 – 3.12	AI Service (YOLOv8)	python.org
Git	ใดก็ได้	Clone โปรเจกต์	git-scm.com
Firebase CLI	13+	Deploy Functions + Rules	<code>npm i -g firebase-tools</code>
Arduino IDE 2.x (ถ้ามี Hardware)	2.x	Flash ESP32 firmware	arduino.cc

ทดสอบได้แม้ไม่มี Hardware: Web App และ Python AI Service สามารถทดสอบครบวงจรได้ผ่าน Firebase Emulator และ REST API โดยไม่ต้องมี ESP32 จริง — ดู Section 5

1 Clone Source Code จาก GitHub

```
# 1. Clone repository
git clone https://github.com/peettanavit/glooprvm.git
cd glooprvm
```

โครงสร้างโฟลเดอร์หลัก

```
glooprvm/
├── web/           ← Next.js Web App
├── ai_server/     ← Python AI + YOLO
├── functions/     ← Firebase Cloud Functions
└── esp32/        ← Arduino Firmware (Master + Slave)
```

2 Web Application (Next.js) — โฟลเดอร์ web/

2.1 ติดตั้ง Dependencies

```
cd web
npm install
```

2.2 ตั้งค่า Environment Variables

```
# สร้างไฟล์ .env.local จาก template
cp .env.example .env.local
```

```
NEXT_PUBLIC_FIREBASE_API_KEY=AIza...
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=glooprvm.firebaseio.com
NEXT_PUBLIC_FIREBASE_PROJECT_ID=glooprvm
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=glooprvm.firebaseio.com
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=...
NEXT_PUBLIC_FIREBASE_APP_ID=1:...
```

รับค่า Firebase Config ได้จาก: Firebase Console → Project Settings → Your apps → Web app → Config snippet

2.3 รันในโหมด Development

```
npm run dev
# เปิด browser ที่ http://localhost:3000
```

2.4 Build สำหรับ Production

```
npm run build
npm run start
```

หน้าที่ทดสอบได้

ROUTE	ฟังก์ชัน
/login	สมัครสมาชิก / เข้าสู่ระบบ
/dashboard	สถานะเครื่อง + คะแนน real-time
/profile	ประวัติการรีไซเคิล
/rewards	แลกรางวัลด้วยคะแนน
/admin	จัดการระบบ (Admin only)

3 Python AI Service (YOLOv8) — โฟลเดอร์ ai_server/

3.1 สร้าง Virtual Environment

```
cd ai_server

# Windows
python -m venv venv
venv\Scripts\activate

# macOS / Linux
python3 -m venv venv
source venv/bin/activate
```

3.2 ติดตั้ง Libraries

```
pip install -r requirements.txt

# Libraries หลัก:
# ultralytics (YOLOv8), opencv-python-headless,
# firebase-admin, fastapi, uvicorn, python-dotenv
```

3.3 รว Service Account Key

```
# วางไฟล์ glooprvm-xxxx.json ใน ai_server/
# ดาวน์โหลดจาก Firebase Console →
# Project Settings → Service Accounts → Generate Key
```

3.4 สร้างไฟล์ .env

```
FIREBASE_SERVICE_ACCOUNT=./glooprvm-xxxx.json
FIREBASE_STORAGE_BUCKET=glooprvm.firebaseio.com
MACHINE_ID=Gloop_01 AI_CONFIDENCE_THRESHOLD=0.50
AI_SAFETY_LOCK_THRESHOLD=0.30 CAP_WAIT_SECONDS=2.0
```

3.5 รว YOLO Models

```
ai_server/models/
├─ label_model.pt ← Primary (label camera)
└─ cap_model.pt ← Validator (cap camera)
```

3.6 รัน AI Service

```
# วิธี 1 - listener เดี่ยว
python listener.py

# วิธี 2 - FastAPI พร้อม /health endpoint
python api.py
# หรือ
uvicorn api:app --host 0.0.0.0 --port 8000
```

ทดสอบ AI ผ่าน REST: เปิด http://localhost:8000/health เพื่อตรวจสอบสถานะ, /status เพื่อดูผลการวิเคราะห์ล่าสุด

4 Firebase Cloud Functions — โฟลเดอร์ functions/

```
cd functions
npm install
```

หมายเหตุ: Cloud Functions ต้อง Deploy ขึ้น Firebase จริง (ไม่รัน local) เพราะ ESP32 เรียกผ่าน URL สาธารณะ ใช้ Firebase project glooprvm ซึ่งมีอยู่แล้ว —

```
# Login Firebase (ครั้งแรก)
firebase login

# Deploy Cloud Functions
firebase deploy --only functions

# Deploy Firestore Rules + Storage Rules
firebase deploy --only firestore:rules,storage
```

5 ESP32 Firmware — โฟลเดอร์ `esp32/` (ต้องมีบอร์ดจริง)

5.1 ติดตั้ง Arduino IDE และ Board Package

- 1 เปิด Arduino IDE 2.x → **File** → **Preferences**
เพิ่ม Board Manager URL:
`https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json`
- 2 **Tools** → **Board** → **Boards Manager** → ค้นหา `esp32 by Espressif Systems` → Install
- 3 เลือก Board: **Tools** → **Board** → **ESP32S3 Dev Module**
- 4 ติดตั้ง Library: **Sketch** → **Include Library** → **Manage Libraries** → ค้นหา `ArduinoJson` v7+ → Install

5.2 ตั้งค่า Credentials

```
# Master ESP32
cp esp32/config.example.h esp32/config.h
# แก้ไข config.h:
# WIFI_SSID, WIFI_PASSWORD
# FIREBASE_API_KEY, FIREBASE_PROJECT_ID
# MACHINE_EMAIL, MACHINE_PASSWORD
# CF_UPLOAD_URL (URL ของ Cloud Function)

# Slave ESP32
cp esp32/config_slave.example.h esp32/config_slave.h
# แก้ไขเหมือนกัน (ใช้ account แยกกัน)
```

5.3 Upload และ Monitor

```
# เปิดไฟล์ใน Arduino IDE
esp32/Master_ESP32/Master_ESP32.ino
esp32/Slave_ESP32/Slave_ESP32.ino

# Upload → ดู Serial Monitor ที่ 115200 baud
```

Firebase Auth สำหรับเครื่อง: สร้าง email แยกต่างหากสำหรับแต่ละบอร์ด โดย email ต้องขึ้นต้นด้วย `machine-` เช่น `machine-gloop01@gmail.com`

6 ⚡ ทดสอบระบบโดยไม่มี ESP32 Hardware (Simulation Mode)

สามารถทดสอบ Web App + AI ได้ครบวงจรโดยไม่ต้องมีบอร์ด ESP32 — ใช้วิธีต่อไปนี้จำลองการทำงานของเครื่อง

วิธีที่ 1 — แก้ Firestore ด้วยมือ (Firebase Console)

- 1 เปิด **Firebase Console** → **Firestore** → **machines** → **Gloop_01**
- 2 สมัครสมาชิก/Login บน Web App (`localhost:3000/login`) → กด **เริ่มใช้เครื่อง** → `status` จะเปลี่ยนเป็น `READY`
- 3 ใน Firestore Console แก้ field `status` เป็น `PROCESSING` และ `session_score` เพิ่มขึ้น → Web App จะแสดงผลทันที (real-time listener)

วิธีที่ 2 — ทดสอบ AI โดยอัปโหลดภาพผ่าน curl / Python

```
# ตรวจสอบว่า AI service รันอยู่ (python api.py)
# ดูลสถานะ service
curl http://localhost:8000/health
curl http://localhost:8000/status

# จำลองขบวนการตรวจ: เขียน status = "ready" ใน Firestore ด้วยมือ
# แล้วใส่ label_storage_path ขึ้นไปที่ภาพวาดใน Firebase Storage
# AI listener จะดึงภาพมา inference และเขียนผลกลับอัตโนมัติ
```

วิธีที่ 3 — ใช้ Firebase Emulator (Local ทั้งหมด ไม่ต้อง Internet)

```
# ติดตั้ง emulator suite
firebase emulators:start --only firestore,auth,storage,functions

# เปลี่ยน web/.env.local ให้ใช้ emulator
NEXT_PUBLIC_USE_EMULATOR=true # (ถ้า app รองรับ)

# รัน web app ตามปกติ - จะเชื่อมกับ emulator แทน Firebase จริง
```

7 ลำดับการทดสอบที่แนะนำ (End-to-End)



COMPONENT	คำสั่งรัน	ทดสอบที่
Web App	cd web && npm run dev	http://localhost:3000
AI Service	cd ai_server && python api.py	http://localhost:8000/health
Cloud Functions	firebase deploy --only functions	Firebase Console → Functions
ESP32 Master	Arduino IDE → Upload Master_ESP32.ino	Serial Monitor 115200
ESP32 Slave	Arduino IDE → Upload Slave_ESP32.ino	Serial Monitor 115200

Firebase Project ID: glooprvm | **GitHub:** <https://github.com/peettanavit/glooprvm> | YOLO models (label_model.pt , cap_model.pt) อยู่ใน ai_server/models/ — ขนาดใหญ่ ติดตาม Git LFS